BÀI TẬP GIỮA KỲ · NHÓM 15 · 5 SINH VIÊN

Ăn hết thức ăn rồi về đích

UCS và A*: vị trí chưa phải toàn bộ trạng thái

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao.

Giảng viên: TS. Nguyễn Bích Vân IAI–UET–VNU.

Nhóm 15: Nguyễn Quốc Hiếu, Lê Đức Huy,Dương Nhật Huy, Phan Ngọc Huy, Hoàng Trung Kiên.

Hạn nộp bài: hết ngày 29-10-2026.

Bảo vệ: 15–20 phút/nhóm.

Mã mẫu hoàn chỉnh, chạy được ngay. Được dùng nguyên phần thuật toán, thay dữ liệu rồi chạy. Nhóm phải trình bày bằng lời của mình, tính tay trước khi đối chiếu, tự viết kiểm thử và giải thích kết quả. Không bắt buộc giao diện đồ họa hoặc thư viện ngoài.

Bài toán

Pacman đi trong mê cung tĩnh, ăn tất cả các viên F rồi kết thúc ở G. Mỗi bước hợp lệ có chi phí 1. Không có ma. Dấu chấm là ô trống; G là vị trí đích, không phải thức ăn. Được đi qua G khi còn F, nhưng chưa được kết thúc.

Câu hỏi trung tâm

Cùng một vị trí nhưng đã ăn những thức ăn khác nhau có phải cùng trạng thái không? Vì sao có thể cần quay lại ô cũ? Heuristic chỉ nhìn thức ăn còn thiếu thông tin gì về nghĩa vụ cuối cùng phải tới G?

Khối lượng và thang điểm

24 cấu hình chung và 12 cấu hình riêng chạy tự động; hai bảng vết tính tay; ít nhất 10 kiểm thử tự viết, mỗi người hai test; báo cáo 6–8 trang chính và 6–8 slide.

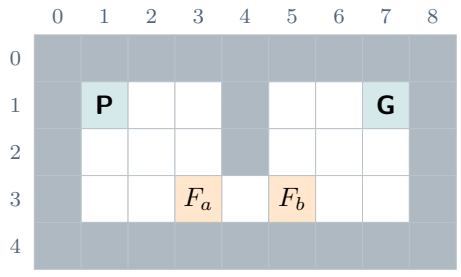
Phần	Nội dung	Điểm
A	Mô hình, thuật toán, chứng minh heuristic	15
B	Tính tay, trạng thái và phản ví dụ	20
C	Python, dữ liệu riêng, trình diễn	20
D	Kiểm thử và nhận xét thực nghiệm	20
E	Báo cáo, tái tạo và cộng tác	10
F	Bảo vệ riêng từng thành viên	15

Điểm sinh viên i là $(G + F_i)/10$, với $G \le 85$ là điểm bài nộp chung và $F_i \le 15$ là điểm bảo vệ cá nhân. Không mặc định năm người có cùng điểm F.

Sản phẩm

Một ZIP có báo cáo PDF, PDF tính tay, slide, các tệp Python chạy được, dữ liệu riêng, JSON/CSV, log kiểm thử, README và bảng đóng góp đúng năm người.

1. Mê cung M0 và mô hình trạng thái



M0

```
%%%%%%%%
%P..%..G%
%...%...%
%..F.F..%
%%%%%%%%
```

Tọa độ (r, c) là hàng, cột; đánh số từ 0 ở góc trên trái, hàng tăng xuống dưới. Mỗi lần đi một ô theo R/D/L/U (phải/xuống/trái/lên); không đi chéo, không quấn mép, không xuyên tường. Không có hành động đứng yên.

Trạng thái đủ thông tin

Gọi F_0 là tập thức ăn ban đầu. Trạng thái $s = (p, R)$ lưu vị trí p và tập $R \subseteq F_0$ thức ăn **còn lại**. Đi vào ô q cập nhật:

$$(p, R) \longrightarrow (q, R \setminus \{q\}), \quad c(s, s') = 1.$$

Điều kiện đích: $p = G$ **và** $R = \varnothing$. Ăn tự động ngay khi bước vào F, không tốn thêm bước. Thức ăn đã ăn không xuất hiện lại khi quay về ô cũ.

Ba cấu hình cần so sánh

$$h_F(p, R) = \begin{cases} \max_{f \in R} d(p, f), & R \neq \varnothing, \\ 0, & R = \varnothing, \end{cases} \qquad d(x, y) = |r_x - r_y| + |c_x - c_y|.$$

$$h_V(p, R) = \max\left(\{d(p, G)\} \cup \{d(p, f) + d(f, G) : f \in R\}\right).$$

UCS dùng $h = 0$; A_F dùng h_F; A_V dùng h_V. Hàm h_V là cận "phải qua từng thức ăn rồi tới G", không phải tổng khoảng cách qua mọi thức ăn.

> Khi $R = \varnothing$, $h_F = 0$ nhưng chưa chắc đã tới G; $h_V = d(p, G)$. Không gọi max trên tập rỗng; không dùng tập F ban đầu thay cho tập còn lại.

Mã hóa trong Python

Sắp F theo hàng rồi cột, gọi là $a, b, \ldots$. Bit 1 là **chưa ăn**. Trạng thái là `(r,c,mask)`. Trên M0: $a = (3, 3)$, $b = (3, 5)$; các tập $ab, a, b, \varnothing$ có mask $3, 1, 2, 0$. Khóa của `best_g` và `parent` phải là toàn bộ bộ ba, không chỉ tọa độ.

2. Thuật toán, quy ước và tính tay B1

Quy ước để tái tạo kết quả

1. RDLU hoặc DRUL áp dụng ở **mọi trạng thái**. Phần tính tay dùng RDLU.
2. Heap UCS ưu tiên (g, id); heap A* ưu tiên $(g+h, \mathrm{id})$. Id tăng theo lần chèn; đồng hạng thì FIFO, không ngầm ưu tiên tọa độ.
3. Chỉ cập nhật g và cha khi gặp đường **rẻ hơn hẳn**. Bản ghi cũ còn trong heap, bị bỏ khi lấy ra nếu $g \neq \texttt{best_g}[s]$.
4. Sau lấy bản ghi hợp lệ, kiểm đích rồi kiểm ngân sách, sau đó mới mở rộng. Không dừng khi vừa sinh G; đích được lấy ra nhưng không mở rộng.

Trường	Quy ước
`popped`	Số lần lấy hợp lệ, gồm đích hoặc trạng thái dừng vì ngân sách.
`expanded`	Số lần thực sự sinh trạng thái con, không tính đích.
`generated`	Số chuyển trạng thái hợp lệ được xét, kể cả con đã gặp bị loại.
`frontier_peak`	Số bản ghi lớn nhất trong heap, có bản ghi cũ; không phải byte RAM.
found/no_path/limit	Có lời giải / đã duyệt hết không có lời giải / hết ngân sách, chưa kết luận.

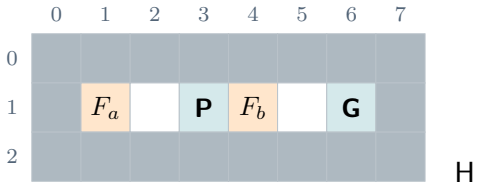
A. Trình bày thuật toán và lập luận – 15 điểm

1. **A1 – 4 điểm.** Mô tả trạng thái, hành động, chuyển, đích và chi phí. Với V ô đi được, m thức ăn, giải thích cận $V2^m$ cho số trạng thái; tại sao đây không nhất thiết là số trạng thái tới được?
2. **A2 – 6 điểm.** Viết mã giả UCS và A*, nêu heap, g, cha, bản ghi cũ và dựng đường. Phân tích thời gian/bộ nhớ theo đồ thị trạng thái mở rộng V', E'; không nhầm V' với số ô. Giải thích vai trò của tập thức ăn trong khóa.
3. **A3 – 5 điểm.** Chứng minh h_F, h_V chấp nhận được, nhất quán và $h_V \geq h_F$. Xét riêng cạnh ăn một F, kể cả viên cuối. Tường có làm mất cận không? Heuristic lớn hơn có bảo đảm luôn chạy nhanh hơn hẳn không?

B1. M0: mô hình và đường hợp lệ – 4 điểm

Xác định P/G/F, số ô đi được và cận số trạng thái. Tính h_F, h_V ở đầu và hai trạng thái con hợp lệ. Tìm đường ăn đủ rồi tới G; ghi dãy hành động, thời điểm ăn từng viên, chi phí. Chứng minh tối ưu bằng cận dưới đạt được, không chỉ dựa vào đầu ra chương trình.

3. Tính tay: quay lại, cập nhật thức ăn và cận



H

H có P tại $(1,3)$, $a=(1,1)$, $b=(1,4)$, G tại $(1,6)$. Trong vết có thể viết $(c;R)$ vì hàng luôn là 1; dấu "–" chỉ tập rỗng.

B2. Sáu lần lấy đầu tiên trên H – 8 điểm

1. Làm hai bảng riêng cho UCS và A_V, mỗi bảng sáu lần lấy hợp lệ. Ghi trạng thái lấy ra, g, h, f, con mới, cha và **toàn bộ OPEN sau xử lý** theo thứ tự sẽ lấy. Với UCS dùng $h=0$.
2. Chỉ ra lần đầu hai thuật toán lấy khác trạng thái; giải thích bằng khóa ưu tiên. Sinh con R trước có buộc A* lấy R trước không?
3. Chỉ đường hoàn chỉnh (không cần chép hết OPEN), tính chi phí và chứng minh tối ưu. Chỉ rõ một ô quay lại nhưng ở trạng thái mới.

B3. Ba lỗi biểu diễn và dừng – 5 điểm

1. Trên H, một bạn chỉ dùng `visited={(r,c)}` và đánh dấu khi phát hiện. Chỉ ra ô cần quay lại; vì sao cách này có thể làm báo không có lời giải? Viết một khẳng định kiểm thử với kỳ vọng tính tay.
2. Trên hành lang `P..G..F`, Pacman có được dừng ở lần đầu qua G không? Tính chi phí tối ưu ăn F rồi về G và số lần ghé G trên đường ấy.
3. Sau khi ăn viên cuối nhưng còn cách G ba bước theo đường thẳng: h_F, h_V và điều kiện đích là gì? Chỉ lỗi của `if h == 0: return success`.

B4. Kiểm tra hai heuristic sai – 3 điểm

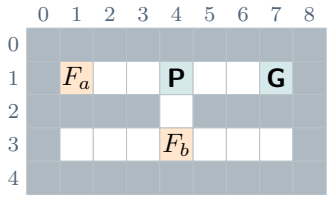
Xét hành lang `PFFG`, mỗi bước giá 1.

1. Tính $h_{\text{sum}} = d(P,G) + \sum_{f\in R} d(P,f)$ ở đầu và so với chi phí tối ưu. Phép cộng này có là cận dưới có bảo đảm không?
2. Một bạn tính h_F bằng tập F **ban đầu** ở mọi trạng thái. Tính hàm ấy khi đã ăn hết, đứng tại G; kiểm điều kiện cần của heuristic chấp nhận được tại đích.
3. Phân biệt no_path và limit. Ngân sách 0 nhưng đầu đã thỏa cả hai điều kiện đích thì cần trả gì?

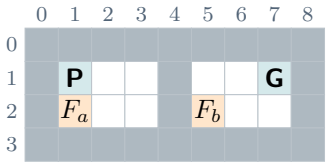
Nộp TinhTay.pdf riêng, có phép tính và bảng bằng bút/bút điện tử. Không chụp output thay cho tính tay. Ghi dự đoán trước khi đối chiếu bằng chương trình.

4. Python, dữ liệu chung và riêng – 20 điểm

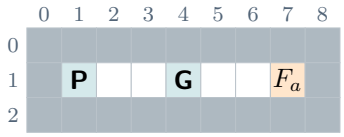
M1: các nhánh phải quay lại



M3: thức ăn bị ngăn cách



M2: qua G rồi trở lại



C1. Chạy và giải thích mã – 8 điểm

Chạy M0–M3, mỗi bản đồ với UCS/A_F/A_V và RDLU/DRUL: **24 cấu hình**. Chỉ sáu đoạn mã: đọc bản đồ; khởi tạo mask; sinh con/xóa bit; heap và g; điều kiện đích; dựng đường. Mỗi đoạn cần tên tệp, hàm và lời giải thích.

```
python run.py --batch --tag bat_buoc
python run.py --case H --algo ucs --trace 6 --tag tay_H_UCS
python run.py --case H --heuristic via --trace 6 --tag tay_H_Avia
```

C2. Hai bản đồ riêng – 6 điểm

Tạo N1/N2 trong `du_lieu_nhom.json`, mỗi bản đồ 5–8 hàng, 9–12 cột, đúng P/G và ba F. N1 có lời giải, đường được chọn phải quay lại ít nhất một ô. N2 **tới G được nếu bỏ yêu cầu thức ăn**, nhưng ít nhất một F không tới được từ P. Chỉ rõ miền bị tách trên hình.

```
python tao_du_lieu.py --seed 1526
python run.py --data du_lieu_nhom.json --batch --tag tu_tao
```

Được dùng bộ sinh với seed riêng, nhưng phải đọc, kiểm, giải thích dữ liệu. Lưu seed, bản đồ và đường N1. Hai bản đồ, ba cấu hình, hai thứ tự: **12 cấu hình riêng**. Không dùng lại M0–M3.

C3. Đối chiếu và trình diễn – 6 điểm

Đối chiếu hai trace với B2; chỉ hai trạng thái cùng tọa độ khác mask. Chạy `-step`, dự đoán rồi Enter. Thay một F hoặc chuyển G trên bản sao bản đồ, dự đoán thông tin phải cập nhật rồi chạy lại; lưu minh chứng riêng, không ghi đè N1/N2.

5. Kiểm thử và nhận xét – 20 điểm

D1. Tối thiểu mười kiểm thử riêng – 8 điểm

Viết `test_nhom.py`; mỗi thành viên ít nhất hai test. Có ba kỳ vọng suy ra bằng tay và chỉ trang trong TinhTay.pdf. Bao phủ các nhóm dưới đây; một test có thể kiểm nhiều thuộc tính liên quan:

1. Ăn F tự động; mask chỉ mất bit, không thêm lại; quay lại cùng ô sau ăn là trạng thái khác.
2. Qua G khi còn F; ăn hết chưa tới G; không có F ban đầu; đầu đã hoàn tất với ngân sách 0.
3. G tới được nhưng thức ăn bị ngăn; hết ngân sách; không trả chi phí 0 khi chưa tìm được lời giải.
4. Bản đồ chữ nhật, biên, không chéo/quấn mép/xuyên tường; từ chối dữ liệu sai.
5. Đổi F/G thì ánh xạ bit và heuristic phải đúng dữ liệu mới. Cả hai hàm bằng 0 tại đích thực.
6. Các cấu hình đồng ý về chi phí tối ưu; đường hợp lệ, ăn đủ, kết thúc tại G; không bắt buộc cùng đường.
7. Không sửa đầu vào; trace bật/tắt không đổi kết quả; A* với $h = 0$, cùng quy ước cho vết giống UCS.

Test mẫu không tính vào mười test riêng. Không dùng chính hàm cần kiểm để sinh kỳ vọng duy nhất. Không bỏ test lỗi rồi ghi tất cả đạt. Chạy `python chay_kiem_thu.py` để lưu log thật.

D2. Bảng so sánh – 6 điểm

Tóm tắt 24+12 dòng theo bản đồ và thứ tự; bảng đầy đủ để ở CSV. So sánh status, chi phí, số mở rộng và đỉnh biên. Với $N_{\text{UCS}} > 0$:

$$S_F = \left(1 - \frac{N_{A_F}}{N_{\text{UCS}}}\right) 100\%, \qquad S_V = \left(1 - \frac{N_{A_V}}{N_{\text{UCS}}}\right) 100\%.$$

Mẫu số 0: ghi "không áp dụng". Không bỏ các ca khó hoặc vô nghiệm để làm đẹp số liệu. Đỉnh biên là số bản ghi, không phải RAM byte.

D3. Ba nhận xét có minh chứng – 6 điểm

Mỗi đoạn 80–120 từ: (i) biểu diễn trạng thái và ảnh hưởng số F; (ii) ảnh hưởng đưa G vào heuristic, nhắc đồng hạng và chi phí tính h; (iii) phân biệt thành công, không có lời giải và giới hạn tài nguyên. Dẫn cấu hình, số liệu hoặc test cụ thể.

> Nêu hai giới hạn của thí nghiệm và một lỗi phát hiện hoặc điều hiểu rõ hơn. Không suy A* luôn nhanh hơn từ vài bản đồ. Đo thời gian tùy chọn: tắt trace/step, chạy ba lần trên cùng máy, lấy trung vị; không chấm tốc độ theo máy nhóm khác.

6. Báo cáo, phân công và bảo vệ

E. Báo cáo và khả năng tái tạo – 10 điểm

Báo cáo **6–8 trang nội dung chính**, không tính bìa/phụ lục; chọn mẫu Word hoặc LaTeX, xuất BaoCao.pdf. Có mô hình, mã giả, chứng minh, đối chiếu tay–mã, kết quả và nhận xét kiểm thử. Chuẩn bị **6–8 slide**. Bảng đóng góp ghi đúng năm người và phần việc kiểm được.

Người	Chủ trì	Kiểm tra chéo
1	Trạng thái, mặt nạ, B1 và UCS	Kiểm dữ liệu người 4
2	A*, hai heuristic, vết A* trên H	Đối chiếu UCS người 1
3	Chứng minh, B3–B4 và phản ví dụ	Đối chiếu heuristic người 2
4	Dữ liệu riêng, chạy 24+12 cấu hình	Kiểm tình huống người 3
5	Kiểm thử, tái tạo, tổng hợp số liệu	Kiểm trace và kết quả người 4

Mỗi người viết ít nhất hai test; cả năm cùng viết nhận xét và kiểm tra chéo. Người 5 tích hợp kiểm thử, không làm thay phần test của các thành viên khác. Không giao một người chỉ ghép báo cáo, không tính đóng góp chỉ bằng số dòng code.

Kế hoạch 10 ngày

Ngày 1–2: đọc luật, chạy mẫu, chia việc. Ngày 3–5: tính tay, chứng minh, đọc mã, ít nhất năm test. Ngày 6–7: đủ mười test, 24+12 cấu hình và đối chiếu. Ngày 8–10: báo cáo, slide, tập bảo vệ, giải nén chạy lại và nộp.

F. Bảo vệ 15–20 phút – 15 điểm/người

Khung 18 phút: 5 phút trình bày, 3 phút demo, 10 phút hỏi năm người (2 phút/người). Chấm riêng năm tiêu chí 0–3: mô hình, dự đoán, đọc mã, xử lý thay đổi, giới hạn kết luận. Không yêu cầu học thuộc mã.

Nộp một ZIP chạy lại được

```
Nhom15_AnHetThucAn_GiuaKy/
  BaoCao.pdf        TinhTay.pdf         ThuyetTrinh.pptx
  README_NOP.md     DongGop.md          test_nhom.py
  maze.py           search.py          data_cases.py     run.py
  du_lieu_nhom.json
  output/  # bat_buoc, tu_tao: JSON/CSV; hai trace; log test
```

Giữ các mô-đun được nhập bởi mã. Không nộp môi trường Python, cache. `dong_goi.py` hỗ trợ kiểm thành phần và tạo ZIP, không chấm nội dung báo cáo.

> Giải nén ZIP sang thư mục mới và chạy lại theo README trên máy thành viên khác. Cả năm người xác nhận PDF khớp JSON/CSV/log.